European Journal of Operational Research 70 (1993) 229–243
North-Holland

Theory and Methodology

A branch and bound algorithm for symmetric 2-Peripatetic Salesman Problems

Jeroen B.J.M. De Kort

Operations Research Group, Institute of Actuarial Science and Econometrics, Faculty of Economics and Econometrics, University of Amsterdam, 1018 WB Amsterdam, Netherlands

Received June 1991; revised November 1991

Abstract: The symmetric 2-Peripatetic Salesman Problem (2-PSP) is an extension of the well-known symmetric Traveling Salesman Problem (TSP) in the sense that 2 edge-disjoint Hamiltonian cycles of minimum total length are required in a graph G with n vertices. Lower bound solutions are provided by 2 minimum length edge-disjoint 1-trees which can be constructed in at most $O(n^2 \log n)$ operations if G is complete. Generating upper bound solutions and executing edge elimination tests takes $O(n^2)$ time. A branch and bound algorithm has been developed based on these procedures to solve symmetric problems. Computational results are available for Euclidean problems up to 60 vertices and problems with randomly generated distance matrices up to 130 vertices.

Keywords: Graphs; Heuristics; Lagrangean multipliers; Branch and bound

1. Introduction

Krarup (1975) introduced the K-Peripatetic Salesman Problem, or K-PSP for short, as:

Given a graph $G = (V, E, C)$ with $V = \{1, \ldots, n\}$ the set of vertices, E the set of edges and $C = (c_{ij})_{n \times n}$ the distance matrix, find K edge-disjoint Hamiltonian cycles, $H_1, \ldots, H_K$, of minimum total length.

We will consider 2-Peripatetic Salesman Problems, which extend Traveling Salesman Problems (TSP) in the sense that for the latter only one Hamiltonian cycle of minimum total length is required.

Several practical applications of the 2-PSP exist. We mention two of them.

In some cases the components on a chip have to be connected by a cycle of minimum total length to minimize the production costs. In fact an optimal TSP solution is required. Due to developments in technology, the capacity of links between two components has drastically increased, thereby increasing the potential damage caused by a link failure. The reliability of the chip is substantially improved by construction of a second cycle that is link-disjoint from the first. Again, to reduce production costs the total length of both cycles has to be minimized.

A second application can be found in job scheduling problems. Consider a one-machine problem where n jobs with identical processing times have to be scheduled twice on the machine. The second

Correspondence to: Jeroen B.J.M. De Kort, Operations Research Group, Institute of Actuarial Science and Econometrics, Faculty of Economics and Econometrics, University of Amsterdam, Reetersstraat 11, 1018 WB Amsterdam, the Netherlands

0377-2217/93/$06.00 © 1993 – Elsevier Science Publishers B.V. All rights reserved

operation can only be executed if all jobs have undergone their first operation. The switch-over time or cost between job i and job j is represented by c_{ij}. Suppose that job i can be succeeded by job j at most once ($i,j = 1,\ldots,n$), for instance due to technological constraints. If the objective is to complete all jobs as early or as cheaply as possible, then the scheduling problem can be formulated as a 2-PSP.

To formulate the 2-PSP mathematically, we introduce 0–1 decision variables $x_{ij}(k)$ that are equal to 1 if edge $[i, j]$ is in the k-th Hamiltonian cycle and 0 otherwise. Then the 2-PSP can be modeled as:

(2-PSP)

$$\text{Min} \quad \sum_{k=1}^{2} \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}(k)$$

$$\text{s.t.} \quad \sum_{i=1}^{n} x_{ij}(k) = 1, \quad k = 1, 2, \quad j \in V, \tag{1}$$

$$\sum_{j=1}^{n} x_{ij}(k) = 1, \quad k = 1, 2, \quad i \in V, \tag{2}$$

$$\sum_{i,j \in S} x_{ij}(k) \leq |S| - 1, \quad k = 1, 2 \quad \forall S \subset V, \quad S \neq V, S \neq \emptyset, \tag{3}$$

$$x_{ij}(1) + x_{ij}(2) \leq 1, \quad i,j \in V, \tag{4}$$

$$x_{ij}(k) \in \{0, 1\}, \quad k = 1, 2, \quad i,j \in V. \tag{5}$$

The objective function in (2-PSP) minimizes the total length of the Hamiltonian cycles. The *degree constraints* (1) and (2) require that a vertex is visited exactly once in each cycle. Inequalities (3) are the *subtour elimination* and (4) the *edge-disjointness constraints*. Similar as for the TSP (Langevin et al., 1990), alternative models for the 2-PSP exist, such as multi-commodity flow formulations.

Using the NP-hardness of the Hamiltonian Path (HP) problem, the NP-hardness of the 2-PSP is easily shown by transforming an instance for the HP problem into an instance for the 2-PSP such that there exists a solution for the 2-PSP if and only if there is a solution for the HP problem. Consequently, the HP problem is a special case of the 2-PSP so that latter is NP-hard. See De Kort (1992) for a complete proof.

In what follows only 2-Peripatetic Salesman Problems are considered with a symmetric distance matrix and with $|E| = \frac{1}{2}n(n-1)$. In order to solve such instances a branch and bound algorithm has been developed. Section 2 is dedicated to the Lagrangean relaxation applied to the degrees constraints (1) and (2) to derive lower bound solutions. Section 3 concentrates on two heuristics to obtain upper bound solutions. Related computational results are available for problems up to 130 vertices. Then in Section 4 the branching rules of the algorithm are given. As outlined in Section 5 the gap between lower and upper bound values can be exploited by a sensitivity analysis on the edges. Computational results of the algorithm for Euclidean and random table problems up to 130 vertices are given in Section 6, followed by some concluding remarks.

2. Construction of lower bound solutions for the 2-PSP

Since any solution of the 2-PSP consists of 2 edge-disjoint 1-trees but not the other way around, a lower bound solution for the 2-PSP is obtained by constructing 2 edge-disjoint spanning trees of minimum total length on $n-1$ vertices, augmented with four shortest edges incident to the remaining vertex. Since computations indicate that the results are insensitive to the choice of the $n-1$ vertices, the trees are constructed on $\{2,\ldots,n\}$.

Alternative lower bound solutions can for instance be derived by solving a Linear Assignment Problem and a Minimum Spanning Tree problem to determine a lower bound solution for each Hamiltonian cycle separately. Furthermore, as described in De Kort (1991a) the subtour elimination constraints can be

relaxed to yield a Capacitated Transportation Problem, solvable in polynomial time by the Stepping Stone method. Computational results, however, show the superiority of the spanning tree approach exploited here.

To solve the disjoint spanning tree problem the algorithm of Roskind and Tarjan (1985) is implemented, that determines K edge-disjoint spanning trees of minimum total length, $T_1, \ldots, T_K$, utilizing a greedy approach. The worst-case running time is dominated by an initial $O(n^2 \log n)$ routine that sorts the edges according to their length. After sorting, construction of the trees requires at most $O(n^2)$ operations if K is fixed.

To improve the lower bound solutions two procedures are used. First the lower bound value is lifted by incorporating Lagrangean multipliers for the combined degree constraints in the objective function. With $\pi = (\pi_1, \ldots, \pi_n) \in \mathbb{R}^n$ the Lagrangean multipliers, the remaining model can be stated as:

(2-PSPLR)

$$\max_{\pi} \min_{x} \left\{ \sum_{k=1}^{2} \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}(k) + \sum_{i=1}^{n} \pi_i (d(i) - 4) \right\}$$

subject to $X = (x_{ij})_{n \times n}$ represents 2 edge-disjoint 1-trees.

According to the standard Lagrangean theory (Geoffrion, 1974), the objective value of (2-PSPLR) is a lower bound value for the 2-PSP. The multiplier adjustment method of Volgenant and Jonker (1982) was used to update the Lagrangean multipliers after determination of the 2 edge-disjoint 1-trees.

Secondly the Lagrangean multipliers have been set up using the following heuristic rule. First a TSP lower bound solution is obtained by generating a minimum 1-tree and updating the multipliers according to the given formula during n iterations. The edges occurring in the best lower bound solution are deleted from the graph, the associated multipliers are stored and the procedure is repeated once. Then the initial value of the Lagrangean multipliers is the average of the stored multipliers. This heuristic decreases the lower bound gap on average from 0.4% to 0.15%, while a slight gain in computer time is attained as well.

If all vertices have degree 4 in a solution of (2-PSPLR) the associated objective value cannot benefit from a multiplier adjustment, since the last term of the objective function is 0. However, such a 4-regular graph does not necessarily contain a 2-PSP solution, since a solution may occur with $d_1(i) = 3$ and $d_2(i) = 1$ for some vertices $i \in V$, where $d_k(i)$ denotes the degree of vertex i in 1-tree k ($i \in V$, $k = 1, 2$). The following theorem holds.

Theorem. *Finding 2 edge-disjoint Hamiltonian cycles on a 4-regular graph is* NP*-complete.*

Proof. First the NP-completeness of the Hamiltonian Cycle (HC) problem on a 4-regular graph is shown by contradiction. If a polynomial algorithm exists for this problem, then the algorithm of Figure 1 is a polynomial algorithm for the HC problem on a 3-regular graph. Since such a graph contains an even

---

Algorithm 3-REGULAR
Input: Graph $G = (V, E)$, with $|V| = n$ and $d(i) = 3$ for $i = 1, \ldots, n$;
Output: A Hamiltonian cycle in G if one exists;
BEGIN
Step 1. {Check a necessary condition}
If no perfect matching M^* in G exists, then go to Step 4;
Step 2. {Create a 4-regular graph}
Duplicate all edges that occur in M^* to obtain a 4-regular graph;
Step 3. {Solve the Hamiltonian Cycle problem}
Apply the algorithm which determines in polynomial time whether a Hamiltonian Cycle on a 4-regular graph exists;
Step 4. Stop;
END.

Figure 1. A polynomial algorithm for the Hamiltonian Cycle problem on a 3-regular graph given a polynomial algorithm for the problem on a 4-regular graph

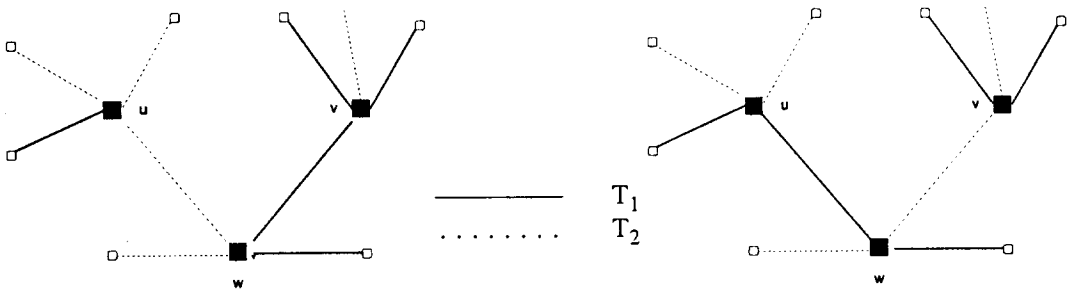


Figure 2. H(4-Reg) exchanges the edges $[u, w]$ in T_2 and $[w, v]$ in T_1 to adjust the degrees of u and v in T_1 and T_2

number of vertices, a necessary condition for the existence of a HC is the existence of a perfect matching, which can be determined by an $O(n^3)$ algorithm. If a perfect matching exists – see Step 1 –, a 4-regular graph is easily constructed in Step 2 by duplicating the edges in the matching and the algorithm for the HC on a 4-regular graph is applied in Step 3. However, Papadimitriou and Steiglitz (1982) have shown that the HC problem on a 3-regular graph is NP-complete.

It follows that no polynomial algorithm exists for the HC problem on a 4-regular graph, unless P = NP. This implies that there is no polynomial algorithm for the 2-PSP on a 4-regular graph, since if such an algorithm would exist the HC on a 4-regular graph would also be solvable in polynomial time.

Given this NP-completeness result, a heuristic has been developed to speed up the branch and bound algorithm by adjusting the degrees of some vertices $i \in V$ with $d_k(i) = 1$, $k = 1, 2$, in a lower bound solution consisting of a 4-regular graph. The heuristic, denoted as H(4-Reg), is based on the observation that for Euclidean problems often vertices with improper degrees in T_1 and T_2 are located within a small area.

First a vertex, say u, with $d_1(u) = 1$ (and $d_2(u) = 3$) is selected. Then H(4-Reg) searches for a path P that satisfies the following properties.

- P consists of edges that are alternating from T_2 and T_1,
- $|P| = 2$, and
- P terminates in a vertex v with $d_1(v) = 3$ (and $d_2(v) = 1$).

If such a path P exists, it can be denoted as $\{u, w, v\}$, with $[u, w] \in T_2$ and $[w, v] \in T_1$. As displayed in Figure 2, H(4-Reg) moves $[u, w]$ to T_1 and $[w, v]$ to T_2, thereby adjusting the degrees of u and v in T_1 and T_2, while the degree of w remains unchanged.

Clearly the heuristic only reshapes the lower bound solution, while the associated lower bound value is not changed. Each time that a 4-regular graph arises in a subset of the branch and bound algorithm H(4-Reg) is activated. Computational experiments indicated that on average the number of vertices with improper degree in T_1 and T_2 reduces about 60%.

3. Upper bound solutions for the 2-PSP

In this section we will report on two heuristics for the 2-PSP that construct the cycles H_1 and H_2, given the minimum 1-trees T_1 and T_2. Furthermore computational results are presented.

Due to space limitations only an outline of the heuristics will be given. De Kort (1992) gives a complete description. Given a lower bound solution T, partitionable into T_1 and T_2, the first heuristic considers T_1 and T_2 simultaneously and the second one successively. The heuristics will be denoted as H(Sim), respectively H(Suc), and are both implemented to run in at most $O(n^2)$ operations.

Unfortunately, the heuristic of Christofides (1976) for the Euclidean TSP that combines a minimum spanning tree and a minimum perfect 1-matching on the vertices with improper degree, cannot be generalized straightforwardly. Indeed, 2 edge-disjoint minimum 1-trees and a minimum perfect 2-matching can be found in polynomial time. However, due to the edge-disjointness requirement, a feasible solution may not exist in the resulting configuration, although there is one in G.

3.1. The Simultaneous Heuristic

H(Sim) consists of 3-stages. The first stage replaces edges that are incident to vertices with improper degree in T. If an edge $e = [i_1, j] \in T$ is such that degree of i_1 exceeds 4, then e is replaced by $[i_2, j]$ with 4 exceeding the degree of i_2. While selecting i_2 the increase in objective value is minimized. If all degrees are equal to 4, this stage can be skipped.

In the second stage a procedure that exploits a similar idea as H(Reg) is utilized to correct for edges incident with vertices having improper degree in T_1 and T_2 (but not in T). Since edges are only moved between T_1 and T_2 and not added to T during this stage, the objective value does not change, but subcycles may be created.

The final stage patches subcycles in T_1 or T_2. Each patching operation contracts two subcycles. To this end the shortest edge in $E \setminus T$ that connects two subcycles, say e, is selected. If edge-disjointness of T_1 and T_2 can be preserved, two subcycles in T_k are patched using e together with another edge in $E \setminus T$ and two edges in T_k, $k \in \{1, 2\}$. Given e, minimization of the increase in objective value is the criterion to select the three additional edges.

3.2. The Successive Heuristic

H(Suc) searches heuristically for a minimum matching on vertices with improper degree in T_1 or T_2, so in some sense it generalizes Christofides' (1976) heuristic for the TSP.

To start with, H(Suc) selects a 1-tree, say T_1 and a vertex i_- with $d_1(i_-) = 1$. By traversing T_1 starting with the edge incident to i_-, vertex i^+ is selected as the first vertex with $d_1(i^+) > 2$. As displayed in Figure 3a, vertices i_1 and i_2 are incident to i^+ in T_1. Figure 3b shows the two alternatives to correct $d_1(i_-)$ and $d_1(i^+)$: Alternative I builds the path $\{i_1, i_-, \ldots, i^+, i_2\}$ in T_1 by adding $[i_1, i_-]$ and omitting $[i_1, i^+]$. Alternative II builds the path $\{i_1, i^+, \ldots, i_-, i_2\}$ in T_1 by adding $[i_-, i_2]$ and omitting $[i^+, i_2]$. The other 1-tree, T_2, is not affected by any alternative. H(Suc) selects the alternative which causes the least increase in objective value, while maintaining edge-disjointness.

However, if there exists a path $\{i_1, i_-, i_2\}$ in T_2, then both alternatives would violate the edge-disjointness restriction. Such a situation occurred many times during our computations with Euclidean problems. In this case H(Suc) moves edges between T_1 and T_2 to enable execution of Alternative I or II.

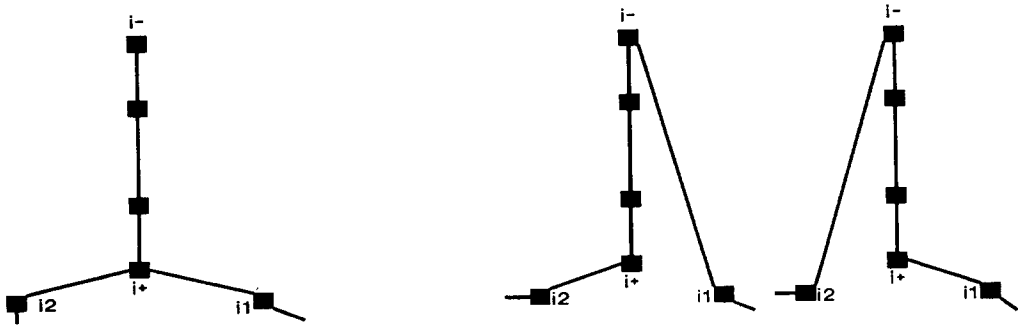


Figure 3. (a) Part of 1-tree T_1 (left half). (b) Two alternatives to correct $d_1(i_-)$ and $d_1(i^+)$. Alternative I to the left and II to the right (right half)

To improve the solutions obtained by H(Sim) and H(Suc) a local search procedure, denoted by LOCSEARCH, is implemented. LOCSEARCH consists of a local improvement and an edge exchange procedure. The improvement procedure focusses on a single Hamiltonian cycle and checks whether a 2-opt or a 3-opt operation (see Croes, 1958, and Lin, 1965) results in a decrease in objective value. The exchange procedure moves edges from H_1 to H_2 or the other way around to enable execution of the improvement procedure while maintaining the edge-disjointness. Although no exhaustive 3-opt procedure is executed much more computer time is required than for each of the upper bound heuristics. Consequently LOCSEARCH is only applied if there is a reasonable change to obtain a better solution. In fact the magnitude of the gap between the generated upper bound value and the best lower bound value is used to determine whether or not LOCSEARCH is activated.

3.3. Computational results for the heuristics

To test the updating scheme for the Lagrangean multipliers and the two heuristics Borland's Turbo Pascal 5.5 has been used to develop a computer program. Computational results are obtained on an IBM PC model 70 with a 80486 processor (and a built in coprocessor) for 45 Euclidean problems and 60 problems with randomly generated distance matrices:

- Five Euclidean problems of 48 vertices and ten problems of 60 vertices from Smith and Thompson (1977). The former class of problems is denoted by ST48, the latter by ST60.
- Ten Euclidean problems for $n = 40, 50, 60$ with randomly generated x,y-coordinates. The classes of problems are denoted by E40, E50, E60.
- Ten problems for $n = 40, 50, 60, 80, 100, 130$ with randomly generated symmetric distance matrices. These classes of problems are denoted by R40, R50, R60, R80, R100 and R130.

The x,y-coordinates and the distance matrices are drawn from a uniform distribution on the integers [1,1000] for which a Pascal version for the PC of a random number routine from the IMSL is used. The seed value of the routine stands at $n^2 + nr$ with $nr \in \{0, \ldots, 9\}$ the sequence number of a problem.

Computational results are obtained by executing the initial ascent, consisting of the setup heuristic for the Lagrangean multipliers followed by $2n$ iterations. In each iteration, a lower bound solution is determined, the multipliers are updated and H(Sim) and H(Suc) executed.

To enable comparison between TSP and 2-PSP instances, the TSP algorithm of Volgenant and Jonker (1982) and Jonker and Volgenant (1984) is used. The initial ascent of this algorithm consists of $3n$ iterations. In each iteration a minimum 1-tree is determined, multipliers for the degree constraints are updated and a heuristic, denoted by H(TSP), is executed.

The first columns in Table 1 and Table 2 display the problem class. Then the gap between the best lower bound and the optimal solution value is given relative to the latter, for the 2-PSP and for the TSP as well. For H(Sim), H(Suc) and H(TSP) the average gap to the optimal value is calculated, again relative to the latter. With respect to the 2-PSP instances, the tables also display the required average CPU times

Table 1
Results for Euclidean problems. Lower and upper bound gaps (percentage relative to the optimal solution) and CPU times in seconds

Size of problems	Lower bound			Upper bound				
	2-PSP		TSP	H(Sim)		H(Suc)		H(TSP)
	gap	CPU	gap	gap	CPU	gap	CPU	gap
E40	0.13	58.3	0.28	0.46	35.3	0.38	8.0	0.04
E50	0.13	116.7	0.42	0.62	32.9	0.35	16.4	0.15
E60	0.27	174.8	0.63	0.57	63.7	0.69	13.0	0.60
ST48	0.08	83.5	0.59	0.55	28.3	0.28	12.3	0.27
ST60	0.21	199.2	0.52	0.54	67.4	0.56	46.6	0.12
average	0.16	126.5	0.49	0.55	45.5	0.45	19.3	0.24

Table 2
Results for random table problems. Lower and upper bound gaps (percentage relative to the optimal solution) and CPU times in seconds

Size of problems	Lower bound			Upper bound				
	2-PSP		TSP	H(Sim)		H(Suc)		H(TSP)
	gap	CPU	gap	gap	CPU	gap	CPU	gap
R40	0.02	53.8	0.19	1.80	17.1	> 5.0	1.0	> 5.0
R50	0.03	130.0	0.10	2.48	47.3	> 5.0	0.9	> 5.0
R60	0.18	199.4	0.15	2.89	38.3	> 5.0	1.2	> 5.0
R80	0.02	254.6	0.21	3.87	45.1	> 5.0	1.4	> 5.0
R100	0.01	305.6	0.15	3.60	59.8	> 5.0	1.8	> 5.0
R130	0.00	609.3	0.17	4.23	84.9	> 5.0	2.4	> 5.0
average	0.04	258.8	0.17	3.14	48.8	> 5.0	1.5	> 5.0

(including about 1% CPU for input/output). The CPU times required by the TSP algorithm are about 10% of the displayed values.

The results indicate that the multiplier adjustment scheme performs well for the 2-PSP: the average lower bound gap does not exceed 0.27%, while the gap for random table problems is less than for Euclidean problems (0.04% to 0.16%). The latter result can be explained by the observation that for all instances the lower bound solution after the initial ascent is a 4-regular (an almost 4-regular) graph, so no (only a slight) increase in objective value is required to obtain a 4-regular graph. For random table problems, this graph usually contains an optimal solution in contrast to Euclidean problems.

It is remarkable that the lower bound gap for the 2-PSP is smaller than for the TSP. Since H(TSP) performs slightly better than H(Suc) and H(Sim) as far as Euclidean problems are concerned, the average initial gaps for the 2-PSP and the TSP are of the same order.

In general H(Suc) outperforms H(Sim), as far as Euclidean problems are considered. Consequently, the initial gaps for Euclidean problems follow from summing the lower bound gap and the gap for H(Suc). For random table problems H(Suc) performs badly: the average gap is more than 5%. This result was expected as H(Suc) generalizes Christofides' heuristic for symmetric *Euclidean* problems. For the same reason H(TSP) generates poor results for random table problems. The CPU time for H(Suc), applied to random table problems, reduces because the time-consuming LOCSEARCH procedure is not executed due to the large upper bound gap. The results indicate that H(Sim) is more capable of solving random table problems, although the quality of the results is lower than for Euclidean problems.

The CPU times of H(Sim) and H(Suc) are about 33% of the total computer time for Euclidean problems and even less for random table problems. Since in our opinion this amount is acceptable in comparison with the generated results, both heuristics were incorporated into the branch and bound algorithm.

4. Branching rules for the 2-PSP

In a good branching scheme the set of feasible solutions, S, is partitioned into a number of subsets, S_1, $S_2, \ldots$, such that the current best lower bound solution is excluded. As usual the branching rules consist of forbidding and requiring edges. A *forbidden* edge does not occur in any optimal solution in a subset generated from S, in contrast to a *required* edge, that occurs in any optimal solution in such a subset. In the sequel the set of forbidden and required edges will be denoted by $F(S)$, respectively $R(S)$.

According to the value of $|R(S)|$, the branching rule has two options:

Branching rule A (BrA): $|R(S)| < 2n$.
If $d(i) = 4$ for all $i \in V$, then the set I is defined as $I = V$. If $d(i) \neq 4$ for some $i \in V$, then the set I is defined as $I = \{i \in V \mid d(i) \neq 4\}$.

A vertex $i^* \in I$ is selected such that the number of required edges incident to i^* is maximal. If there is a further choice, then i^* is selected so that the number of forbidden edges incident to i^* is maximal. Five edges, say $b_1, \ldots, b_5$, that are incident to i^* and not in $F(S)$ are selected to generate at most five disjoint subsets:

$$\begin{aligned}
&R(S_1) = R(S) \cup \{b_1, b_2, b_3, b_4\}, && F(S_1) = F(S) \cup \{b_5\},\\
&R(S_2) = R(S) \cup \{b_1, b_2, b_3\}, && F(S_2) = F(S) \cup \{b_4\},\\
&R(S_3) = R(S) \cup \{b_1, b_2\}, && F(S_3) = F(S) \cup \{b_3\},\\
&R(S_4) = R(S) \cup \{b_1\}, && F(S_4) = F(S) \cup \{b_2\},\\
&R(S_5) = R(S), && F(S_5) = F(S) \cup \{b_1\}.
\end{aligned}$$

Less than five subsets are generated if $b_j \in R(S)$ for some $j \in \{1, \ldots, 5\}$. Computational experiments showed that the results improved by using two selection rules for $b_1, \ldots, b_5$:

1) All edges incident to i^* in T are among $b_1, \ldots, b_5$.
2) $\Delta(b_1) \geq \Delta(b_2) \geq \cdots \geq \Delta(b_5)$, where $\Delta(b_j)$ denotes the increase in objective value if b_j is made a forbidden edge, $j = 1, \ldots, 5$.

Using a depth-first search strategy the idea behind these selection rules is to enlarge the change that better solutions are encountered and to exclude the current best solution.

Branching rule B (BrB): $|R(S)| = 2n$.

To determine whether two Hamiltonian cycles, H_1 and H_2, exist in S the set $R_k(S)$ is defined as the set of required edges in H_k $(k = 1, 2)$. A vertex $i^* \in V$ and a configuration k^* are selected such that $d_{k^*}(i^*) = 3$. If there is a further choice, then i^* and k^* are selected so that the number of edges in $R_{k^*}(S)$ incident to i^* is maximal. Since $d(i) = 4$ for all $i \in V$, four edges incident to i^*, say $b_1, \ldots, b_4$, are present. Without loss of generality, it is assumed that $d_1(i^*) = 3$ and that b_1, b_2 and b_3 are incident to i^* in T_1. Now BrB distinguishes between two cases to generate disjoint subsets S_1, S_2 and S_3 that exclude the current best lower bound solution:

1) If one edge, say b_1, is in $R_1(S)$, then

$$\begin{aligned}
&R_1(S_1) = R_1(S) \cup \{b_2\}, && R_2(S_1) = R_2(S) \cup \{b_3, b_4\},\\
&R_1(S_2) = R_1(S) \cup \{b_3\}, && R_2(S_2) = R_2(S) \cup \{b_2, b_4\},\\
&R_1(S_3) = R_1(S) \cup \{b_4\}, && R_2(S_3) = R_2(S) \cup \{b_2, b_3\}.
\end{aligned}$$

2) If b_1, b_2 and $b_3 \notin R_1(S)$, then

$$\begin{aligned}
&R_1(S_1) = R_1(S) \cup \{b_1, b_2\}, && R_2(S_1) = R_2(S) \cup \{b_3, b_4\},\\
&R_1(S_2) = R_1(S) \cup \{b_1\}, && R_2(S_2) = R_2(S) \cup \{b_2\},\\
&R_1(S_3) = R_1(S), && R_2(S_3) = R_2(S) \cup \{b_1\}.
\end{aligned}$$

The first option leads to a branching into two subsets if $b_4 \in R_2(S)$.

As will be described in the following section, tests have been implemented to extend $R(S)$, $F(S)$ and $R_k(S)$, thereby reducing the number of subsets generated by the algorithm.

5. Edge elimination tests

The gap between lower and upper bound value enables execution of a sensitivity analysis to forbid or require edges. Moreover some tests based on logical reasoning are developed for the same purpose. A rather powerful test to eliminate edges for the TSP is based on the observation that a tour can be optimal only if it is 2- and 3-optimal (Jonker and Volgenant, 1984). In the 2-PSP case however, this test

does not hold since for a single tour neither 2- nor 3-optimality is a necessary condition to obtain optimality. To describe alternatives for the 2-PSP some additional terminology will be used:

- U denotes the best upper bound value and T the best lower bound solution.
- E_s denotes the available edges in S: $E_s = \{[i, j] \in E \mid [i, j] \notin F(S)\}$.
- $W \subseteq V$, $|W| \geq 1$, is a cutset of V, with
  $E_s(W) = \{[i, j] \in E_s \mid i \in W, j \in W\}$;
  $E_s(W : V \setminus W) = \{[i, j] \in E_s \mid i \in W, j \in V \setminus W\}$;
  $|E_s(W : V \setminus W)|$ is the capacity of the cutset that obeys the relation

$$|E_s(W : V \setminus W)| = \sum_{v \in W} E_s(v : V \setminus v) - 2|E_s(W)|.$$

If W is a singleton set, we will use $W = j$ instead of $W = \{j\}$.

As before it will be assumed that G is complete. The results are easily generalized for non-complete graphs. With respect to subsets generated by BrA, Section 5.1 concentrates on tests to find forbidden edges, Section 5.2 on tests to find required edges. Section 5.3 concentrates on tests to find required edges in subsets generated by BrB.

5.1. Forbidden edges in S

To extend the set $F(S)$ without executing a branching, two tests based on logical reasoning and two tests based on the gap between lower and upper bound value have been implemented.

F1. Degree Four Test. If requiring an edge e results in an edge set that cannot contain a feasible solution, e.g., $n - 4$ forbidden or 5 required edges incident to one vertex, then i is a forbidden edge. Implementation of this test is straightforward and requires $O(n^2)$ operations.

F2. Cutset Test. If a cutset W satisfies the following properties for $m \geq 2$:
a) $|E_s(W : V \setminus W)| = 2m + 1$, and
b) $|R(S) \cap E_s(W : V \setminus W)| = 2m$,
then the unique edge in $E_s(W : V \setminus W) \setminus R(S)$ is a forbidden edge, as the capacity of each cutset is even (and at least 4) in an optimal solution. For $m = 2$, an example is displayed in Figure 4. The edge [1,2] can be forbidden.

Since exponential many cutsets exist for which a) and b) may hold, checking this observation for all cutsets is too time consuming. Therefore a shrinking procedure has been implemented that is based on the following observation. Suppose that a set W exists for which a) and b) hold and that $i \in W$ is such that there exists an edge $[i, j] \in E_s$, with $2|E_s(j : W)| = |E_s(j : V \setminus j)|$. If $j \in W$, then we can shrink i and j into a single vertex. If $j \notin W$, the set W^* is obtained as $W^* = W \cup j$.

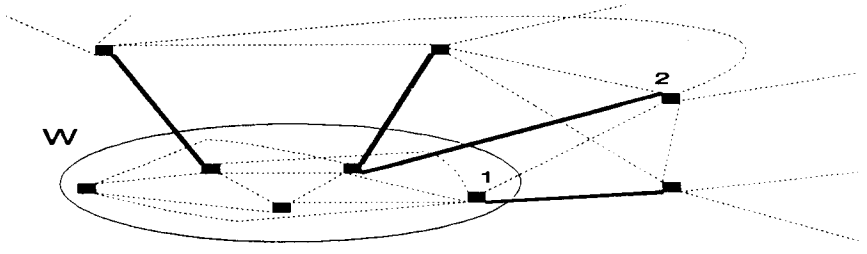


Figure 4. $|E_s(W : V \setminus W)| = 5$ and $|R(S) \cap E_s(W : V \setminus W)| = 4$ (indicated by the bold lines). Consequently $[1,2] \in E_s(W : V \setminus W) \setminus R(S)$ can be forbidden

Now,

$$|E_s(W^*:V\setminus W^*)| = \sum_{v\in W^*} |E_s(v:V\setminus v)| - 2|E_s(W^*)|$$
$$= \sum_{v\in W} |E_s(v:V\setminus v)| + |E_s(j:V\setminus j)| - (2|E_s(W)| + 2|E_s(j:W)|)$$
$$= \sum_{v\in W} |E_s(v:V\setminus v)| - 2|E_s(W)| = 2m+1.$$

As a consequence W^* satisfies a). Starting with two vertices $W=\{i_1, i_2\}$, for which $E_s(W:V\setminus W)=2m+1$, the shrinking procedure is executed until a set W is encountered that satisfies condition a) and b) or until one vertex remains in the graph. To restrict the computer time we have chosen $m \leq 4$.

F3. Pricing Out Test. If requiring an edge $e_{\text{out}} \notin T$ results in an increase in objective value that exceeds the gap between lower and upper bound value, e_{out} is a forbidden edge. Stated more formally, let $e_{\text{in}} \in T$ denote an edge that leaves the lower bound configuration if e_{out} is forced in. Then e_{out} is a forbidden edge if

$$c(T) - c(e_{\text{in}}) + c(e_{\text{out}}) > U - 1. \tag{6}$$

The value $U-1$ is used, since we are satisfied with finding only one optimal solution. In De Kort (1991b) it is shown that the partitioning algorithm of Edmonds (1965) can be used to determine e_{in} for all edges in $E_s\setminus T$ in at most $O(n^2)$ operations.

The test can be improved by raising the left hand side of (6). Therefore we compute for each vertex $i \in V$,

$$\gamma(i) = \min\{c_{st} - c_{ij} \mid c_{st} \geq c_{ij}, \text{ with } s\neq i,\ j;\ t\neq i,\ j;\ [s,t]\in E_s\setminus T,\ [i,j]\in T$$
$$\text{and } [s,t] \text{ can replace } [i,j] \text{ in } T\}.$$

Consider an edge $e_{\text{out}} = [u, v] \in E_s\setminus T$. If π_u is raised to $\pi_u + \gamma(u)$, then, by definition of $\gamma(u)$, T does not change, while the left-hand side of (6) will go up $\gamma(u)\cdot(d(u)-4)+\gamma(u) = \gamma(u)\cdot(d(u)-3)$, if $c(e_{\text{in}})$ is not affected by changing π_u. Otherwise the improvement will be less, but may be still positive. The same holds for π_v and $\gamma(v)$.

For all $i \in V$, the value $\gamma(i)$ is computed simultaneously with the construction of T_1 and T_2 exploiting the following idea. If during the construction of T_1 and T_2 an edge $[u, v]$ not yet in $T_1 \cup T_2$ does not create a cycle in T_k, then it is added in T_k, $k \in \{1, 2\}$. In the other case the edges on these cycles are moved to the other (sub-)tree and the procedure is repeated: if one of moved edges creates a cycle, the edges occurring on the cycles are moved. The procedure finishes if an edge is found that does not create a cycle or if no more edges can be moved. In the latter case the moved edges clearly can be replaced by $e_{\text{out}} = [u, v]$. If $c_{uv} - c_{ij} < \gamma(i)$ ($u \neq i$ and $v \neq i$) for such an edge $[i, j] \in T$, then $\gamma(i)$ is adjusted.

F4. Alternative Lower Bound Test. Let T denote 2 edge-disjoint spanning trees on n vertices of minimum total length. T^+ is obtained by augmenting T with the shortest edge in $E_s\setminus T$. If for an edge $e_{\text{out}} \in E_s\setminus T^+$,

$$c(T^+) + c(e_{\text{out}}) > U - 1, \tag{7}$$

then e_{out} is a forbidden edge.

This test is easily implemented by constructing 2 edge-disjoint spanning trees of minimum total length on n vertices (instead of on $n-1$ vertices). The left hand side of (7) can be raised by using the term $\gamma(i)$ as before.

5.2. Required edges in S

To require edges in the set S two tests based on logical reasoning and one test based on the gap between lower and upper bound value will be described.

R1. Degree Four Test (compare F1). If forbidding an edge e results in an edge set that does not contain a feasible solution, e.g., $n-4$ forbidden or 5 required edges incident to one vertex, then e is a required edge. R1 is of $O(n^2)$.

R2. Cutset Test (compare F2). If a cutset W satisfies $|E_s(W:V\setminus W)| = 4$, then all edges in $E_s(W:V\setminus W)$ are required. If $|W| = 1$, this test is a special case of R1. The shrinking procedure of the previous section is used to find a set W with the required property.

R3. Pricing In Test (compare F3). If forbidding an edge $e_{in} \in T$ results in an increase in objective value that exceeds the gap between lower and upper bound value, then e_{in} is a required edge. Stated more formally, let e_{out} denote an edge that will enter T if e_{in} is made a forbidden edge. Then e_{in} is a required edge if

$$c(T) - c(e_{in}) + c(e_{out}) > U - 1. \tag{8}$$

Checking (8) for all edges $e_{in} \in T$ can be done in at most $O(n^2)$ operations since the procedure is almost identical to determining the term $\gamma(i)$ – see the previous section. Suppose that an edge e_0 can not be added to the (sub)-trees T_1 or T_2 even if edges are moved between the (sub-)trees. In this case the moved edges for which $c(e_{out})$ has not yet been determined are stored. Now $c(e_{out}) = c(e_0)$ for all stored edges, since there is no shorter edge than e_0 that can replace one of the stored edges in T.

The power of the test can be improved by raising the left-hand side of (8) using the term $\gamma(i)$ that is defined in the previous section.

5.3. Required edges in H_k

To avoid complete enumeration as long as possible while determining whether H_1 and H_2 exist in a 4-regular graph E_s, the following tests have been implemented to require edges in H_k, $k \in \{1, 2\}$.

***R*$_k$1. Degree Two Test.** If two edges incident to one vertex are in $R_k(S)$, the remaining two edges incident to this vertex are in $R_{3-k}(S)$, $k \in \{1, 2\}$.

***R*$_k$2. Chain Test.** Let C_k denote a chain of required edges in H_k and let i and j denote its endpoints. If $|C_k| < n-1$ and $[i, j]$ is a required edge, then $[i, j]$ can be added to $R_{3-k}(S)$, $k \in \{1, 2\}$.

***R*$_k$3. Hamiltonian Path Test.** If a cutset W contains only two vertices that are incident to edges with exactly one endpoint in $V\setminus W$, then any optimal solution contains two edge-disjoint Hamiltonian paths between these two vertices. Stated more formally, if for a cutset $W \subseteq V$, $|\{v \in W: |E_s(v:V\setminus W)| > 0\}| = 2$, then any feasible solution in E_s contains two edge-disjoint Hamiltonian paths in W. Consequently, if two Hamiltonian paths are found in $E_s(W)$, then all edges in $E_s(W)$ can be added to either $R_1(S)$ or $R_2(S)$ and the edges in $E_s(W)$ are excluded from further consideration. This test is based on the observation that an optimal solution in E_s is not necessarily unique.

***R*$_k$4. Cutset Test** (compare F2). If a cutset W satisfies the following properties for $m \geq 2$ and $k \in \{1, 2\}$:
a) $|E_s(W:V\setminus W)| = 2m$, and
b) $|R_k(S) \cap E_s(W:V\setminus W)| = 2m-2$,
then in any feasible solution, H_{3-k} contains the pair of edges in $R_k(S)\setminus E_s(W:V\setminus W)$, since otherwise the vertices in W are visited only once. If $m = 2$, then any feasible solution in E_s contains two

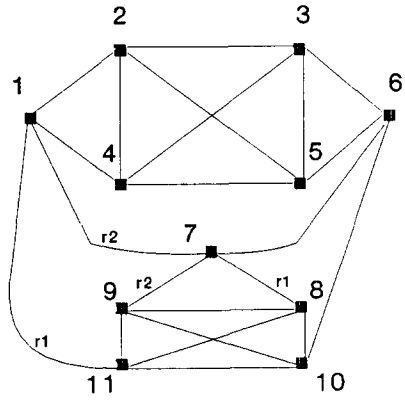


Figure 5. A 4-regular graph on 11 vertices. $R_1(S)$ contains [1,11] and [7,8]; $R_2(S)$ contains [1,7] and [7,9]

edge-disjoint Hamiltonian paths in W. As before, the edges in $E_s(W)$ can be excluded from further considerations.

Execution of the tests is illustrated by the example of Figure 5, a 4-regular graph on 11 vertices. The edges [1,11] and [7,8] are in $R_1(S)$ (indicated by r1), [1,7] and [7,9] in $R_2(S)$ (indicated by r2). Applying the Degree Two Test on vertex 7 results in adding [6,7] to $R_1(S)$. Then choosing $W = \{1, 2, 3, 4, 5, 6\}$ and applying the Hamiltonian Path Test, gives

$$|\{v \in W: |E_s(v: V \setminus W)| > 0\}| = |\{1, 6\}| = 2.$$

Consequently the edges on the path {1, 2, 3, 4, 5, 6} can be added to $R_1(S)$ and the remaining edges with both endpoints in W to $R_2(S)$ without excluding all feasible solutions in E_s. Now the Cutset Test is applied with $W = \{8, 9, 10, 11\}$:

$$|E_s(W: V \setminus W)| = |\{[7,8], [7,9], [6,10], [1,11]\}| = 4$$

and

$$|R_1(S) \cap E_s(W: V \setminus W)| = |\{[7,8], [1,11]\}| = 2,$$

so [6,10] can be added to $R_2(S)$. Furthermore the edges on the Hamiltonian path {8, 9, 10, 11} are added to $R_1(S)$ and the remaining edges with both endpoints in W to $R_2(S)$. Consequently we have

H_1 = 1-2-3-4-5-6-7-8-9-10-11 and H_2 = 1-4-2-5-3-6-10-8-11-9-7.

6. Computational results of the branch and bound algorithm

The procedures described in the previous sections provide a branch and bound algorithm to solve symmetric 2-PSP instances to optimality. The input for the algorithm consists of the problems of Section 3.3. Results for the Euclidean problems are in Table 3 and for the random table problems in Table 4. The TSP branch and bound algorithm of Volgenant and Jonker (1982) and Jonker and Volgenant (1984) is used to enable comparison with TSP results. The TSP algorithm uses fictitious upper bound values for random table problems, since the implemented upper bound heuristic performs poorly on random table problems, as pointed out in Table 2. Clearly, fictitious upper bounds speed up the algorithm, provided they are used properly.

In the first column of Table 3 and Table 4 the problem name is given followed by the number of generated subsets and lower bound solutions. For the 2-PSP, a lower bound solution consists of two edge-disjoint 1-trees, for the TSP of one 1-tree. Besides average values, indicated by μ, the minimum and

Table 3
Results for Euclidean problems solved to optimality. The number of generated subsets, lower bound solutions and required CPU times (in minutes)

Size of problems	# subsets				# lower bound solutions				CPU		
	min	max	μ	μTSP	min	max	μ	μTSP	min	max	μ
E40	1	5021	534	3	80	9353	1122	126	1.1	77.4	10.2
E50	19	19032	3571	17	115	18559	5027	284	1.8	389.4	75.4
E60 [a]	179	7882	2008	26	1053	13093	4759	456	17.7	232.1	79.7
ST48	16	229	73	10	108	768	316	230	1.8	16.5	5.5
ST60 [a]	52	42118	4947	19	267	57605	7258	341	5.7	674.5	90.2

[a] One problem in this class was not solved by the 2-PSP algorithm within 10 hours CPU.

maximum number of subsets and lower bound solutions is displayed for the 2-PSP as well. The three right most columns show the CPU times for the 2-PSP algorithm on an IBM PC with a 80486 processor and built in coprocessor. The TSP algorithm is 10–15 times as fast.

To improve the performance of the algorithm fictitious upper bounds were incorporated. Since the upper bounds heuristics generate satisfactory results for Euclidean problems the fictitious upper bounds for these problems were calculated according to the *quality guarantee method* of Lawler and Wood (1966):

$\text{ubf} = \lambda \cdot U$ where U is the best upper bound value after the initial ascent.

For our purpose, λ has been set to 0.995 and the algorithm terminates either with a new feasible, i.e., optimal solution, or no feasible solution is found. Then, or if $\lambda \cdot U < c(T)$, λ is increased 0.0025 and the procedure repeated.

Regarding the results of Section 3.3, fictitious upper bounds for random table problems are based on the lower bound instead of the upper bound value:

$\text{ubf} = (1 + \mu) \cdot c(T)$ where T is the best lower bound solution.

With $\mu = 0.005$, a solution was found for all instances, so no restarts were required.

Computational results are in Table 5, where the average improvements over the values in Table and Table 4 are displayed. Some conclusions can be drawn.

1) 103 out of the 105 instances were solved to optimality; 71% of the problems was solved within 15 minutes CPU, 88% within 30 minutes and 92% within one hour.
2) Especially for random table problems the required number of subsets, lower bound solutions and CPU times decrease substantially utilizing fictitious upper bounds. With respect to small Euclidean instances (E40) the performance of the algorithm sometimes decreases, since the small initial gap

Table 4
Results for random table problems solved to optimality. The number of generated subsets, lower bound solutions and required CPU times (in minutes)

Size of problems	# subsets				# lower bound solutions				CPU		
	min	max	μ	μTSP [a]	min	max	μ	μTSP [a]	min	max	μ
R40	1	243	67	4	65	610	210	130	0.5	3.9	1.4
R50	1	488	129	2	115	955	327	143	1.0	7.7	3.4
R60	45	495	137	14	136	968	360	301	1.9	9.6	7.5
R80	273	606	390	16	502	1517	912	411	9.5	25.4	16.7
R100	29	284	212	47	178	930	606	961	4.4	18.4	20.5
R130	98	971	312	77	309	2172	828	1624	13.0	70.3	30.0

[a] All problems solved with a fictitious upper bound.

Table 5
Results for Euclidean and random table problems generated by the 2-PSP algorithm with fictitious upper bounds. All values are percentages

Size of problems	Improvement over μ			Size of problems	Improvement over μ		
	subsets	1b sol	CPU		subsets	1b sol	CPU
E40	−11.4	−4.4	−3.0	R40	33.4	28.9	40.1
E50	77.0	71.0	75.2	R50	73.2	51.2	63.4
E60	35.2	55.3	59.0	R60	39.8	32.6	68.1
ST48	−15.3	12.1	35.4	R80	48.7	41.7	54.2
ST60 [a]	23.5	20.8	20.5	R100	2.6	1.5	6.1
				R130	46.7	36.0	30.7

[a] One problem in this class was not solved by the 2-PSP algorithm with ubf, within 10 hours CPU.

causes a restart of the algorithm as no feasible solution is found for $\lambda = 0.995$. For larger Euclidean problems the performance of the algorithm improves in general. For instance after construction of about 32000 lower bound solutions in 23000 subsets, an optimal solution was determined for E609 which was not solved by the original algorithm.

3) As for the TSP, the 2-PSP is easier to solve for random table problems than for Euclidean problems of equal size, although the initial gap for the former class of problems usually exceeds the one for the latter. As pointed out in Section 3.3, this can be explained by the observation that a 4-regular graph, encountered early in the algorithm, contains in 95% (20%) of the cases a feasible solution when random table (Euclidean) problems are considered.
4) The results suggest that for moderate sized Euclidean problems (E50, E60 and ST60) the TSP algorithm benefits from the edge elimination test based on the λ-optimality property of an optimal solution, which cannot be generalized for the 2-PSP. This follows since for these classes the TSP results are clearly superior to those of the 2-PSP.

In view of this result it is remarkable that the average number of generated 1-trees exceeds the average number of generated edge-disjoint 1-trees for large random table problems (100 vertices or more). This observation suggests that it is not hard to find minimum-length edge-disjoint 1-trees providing good lower bound solutions. Therefore the question arises whether the TSP algorithm provides good upper bound solutions by solving the TSP on G, deleting the edges in the optimal solution from G and solving the TSP again. The average upper bound gaps generated this way relative to the optimum are displayed in Table 6, supporting the conjecture that for random table problems the average gap is even smaller than the one generated by our 2-PSP heuristics (see Table 2). Unfortunately, the TSP heuristic has little practical value, since the computation times are about 15–20 times as large as for the 2-PSP heuristic.

Table 6
Results for Euclidean and random table problems. Average upper bound gaps generated by the TSP algorithm

Size of problems	Upper bound gap			Size of problems	Upper bound gap		
	max	min	average		max	min	average
E40	1.23	0.31	0.65	R40	1.96	0	0.81
E50	1.87	0	0.63	R50	1.65	0.03	0.77
E60	1.71	0.27	0.77	R60	2.41	0.21	0.89
ST48	1.62	0.38	0.47	R80	1.18	0.22	0.78
ST60	3.92	0.11	1.02	R100	1.22	0.33	0.73
				R130	1.01	0.39	0.72
average	2.07	0.21	0.71	average	1.57	0.20	0.78

7. Concluding remarks

A branch and bound algorithm was presented to solve symmetric 2-Peripatetic Salesman Problems up to 130 vertices on a personal computer. Lower bound solutions are obtained by generating 2 minimum length edge-disjoint 1-trees and upper bound solutions by two heuristics that are based on the lower bound solution. Computational results indicate that one heuristic generates better results for Euclidean problems and the other for problems with randomly generated distance matrices.

It was shown that finding an optimal 2-PSP solution in a 4-regular graph is NP-complete. Based on this result the 2-PSP is decomposed into two subproblems with separate branching schemes: in first instance, a 4-regular graph is obtained by forbidden and requiring edges; secondly, edges are forbidden and required in a Hamiltonian cycle to determine whether a 2-PSP solution exists in the graph.

A total of 11 edge elimination tests are implemented based on logical reasoning or on a sensitivity analysis that uses the gap between lower and upper bound value. The tests benefit from a good upper bound value, so that fictitious upper bounds improve the performance of the algorithm, especially for random table problems. With respect to Euclidean problems good upper bound solutions are generated by the algorithm itself.

It remains to be elucidated whether an LP approach with a facet generating procedure for the 2-PSP is able to solve larger problems as is the case for the TSP (Crowder and Padberg, 1980).

References

Christofides, N. (1976), "Worst-case analysis of a new Heuristic for the Traveling Salesman Problem", Technical Report, GSIA, Carnegie-Mellon University.

Croes, G.A. (1958), "A method for solving Traveling-Salesman Problems", *Operations Research* 6, 791–812.

Crowder, H., and Padberg, M. (1980), "Solving large-scale Traveling Salesman Problems to optimality", *Management Science* 26, 495–509.

De Kort, J.B.J.M. (1991a), "Lower bounds for symmetric *K*-Peripatetic Salesman Problems", *Optimization* 21, 113–122.

De Kort, J.B.J.M. (1991b), "Sensitivity Analysis for the 2-Peripatetic Salesman Problem", AE Report 4/91, Institute of Actuarial Science and Econometrics, University of Amsterdam, to appear in *Operations Research Letters*.

De Kort, J.B.J.M. (1992), "Upper bounds and lower bounds for the symmetric *K*-Peripatetic Salesman Problem", *Optimization* 22, 357–367.

Edmonds, J. (1965), "Minimum partition of a matroï into independent subsets", *Journal of Research of the National Bureau of Standards B* 69, 67–72.

Geoffrion, A.M. (1974), "Lagrangean relaxation for Integer Programming", *Mathematical Programming Study* 2, 81–114.

Jonker, R., and Volgenant, T. (1984), "Nonoptimal edges for the symmetric Traveling Salesman Problem", *Operations Research* 32, 837–846.

Krarup, J. (1975), "The Peripatetic Salesman and some related unsolved problems", in: B. Roy (ed.), *Combinatorial Programming, Methods and Applications*, Reidel, Dordrecht, Holland 173–178.

Langevin, A., Soumis, F., and Desrosiers, J. (1990), "Classification of Traveling Salesman Problem formulations", *Operations Research Letters* 9, 127–132.

Lawler, E.L., and Wood, D.E. (1966), "Branch-and-bound methods: A survey", *Operations Research* 14, 699–719.

Lin, S. (1965), "Computer solutions of the Traveling Salesman Problem", *Bell System Technical Journal* 44, 2245–2269.

Papadimitriou, C.H., and Steiglitz, K. (1982), *Combinatorial Optimization, Algorithms and Complexity*, Prentice-Hall. Englewood Cliffs, NJ.

Roskind, J., and Tarjan, R.E. (1985), "A note on finding minimum-cost edge-disjoint spanning trees", *Mathematics of Operations Research* 10, 701–708.

Smith, T.H.C., and Thompson, G.L. (1977), "A lifo implicit enumeration search algorithm for the symmetric traveling salesman problem using Held and Karp's 1-tree relaxation", *Annals of Discrete Mathematics* 1, 479–493.

Volgenant, T., and Jonker, R. (1982), "A branch and bound algorithm for the symmetric Traveling Salesman Problem based on the 1-tree relaxation", *European Journal of Operational Research* 9, 83–89.

Volgenant, T., and Jonker, R. (1983), "The symmetric Traveling Salesman Problem and edge exchanges in minimal 1-trees", *European Journal of Operational Research* 12, 394–403.